



DEVNEXES DIGITAL SOLUTIONS

AI / ML INTERNSHIP PROJECT PLANS

Six individual, medium-to-challenging professional projects with complete weekly execution plans

Individual Internship Project Portfolio

Professional project briefs, requirements, execution plans, weekly tasks, quality gates and final submission standards

Issued: July 2026 | Version 1.0

We Don't Just Build, We Solve.



1. Purpose of This Document

This document provides a complete individual-project framework for the Devnexus AI / ML Internship. Each intern should be assigned one project only. The projects are intentionally designed at a medium-to-challenging level so that the final output demonstrates practical engineering ability, professional decision-making and portfolio readiness rather than basic tutorial completion.

The AI/ML track emphasizes responsible model development, evidence-based evaluation and deployable prototypes. Interns are expected to understand the data, establish a baseline, compare approaches, report meaningful metrics and communicate limitations. A polished interface alone is not sufficient without a reproducible machine-learning workflow.

Project assignment rule: Every intern must build the assigned project independently. Collaboration for discussion is allowed, but source code, design files, datasets, reports and deployment work must remain the intern's own. Generic CRUD-based management systems are outside the scope of this internship portfolio.

2. Project Options

1. Devnexus DocuSense - Citation-Aware Document Question Answering Assistant
2. Devnexus TalentMatch - Resume Analyzer and Job Matching Engine
3. Devnexus MeetMind - Meeting Transcript Intelligence Tool
4. Devnexus VisionInspect - Visual Defect Classification and Explainability
5. Devnexus ReviewGuard - Suspicious Review Detection with Explanations
6. Devnexus RecoLab - Hybrid Recommendation Engine with Cold-Start Handling

3. Mandatory Professional Standards

1. Create a separate public or private GitHub repository for the project. The repository name must be professional, must relate to Devnexus and must clearly indicate the project purpose. Example format: Devnexus-ProjectName.
2. Add a complete README containing the problem statement, objectives, feature list, architecture, technology stack, setup instructions, environment-variable instructions, screenshots, testing notes and deployment link.
3. Push code regularly throughout the internship. A single final upload is not acceptable. Commit history must show genuine progress, meaningful milestones and professional commit messages.
4. AI tools such as ChatGPT, Gemini, GitHub Copilot or other assistants may be used for guidance. However, every generated line of code, design decision and written explanation must be reviewed, understood, tested and improved by the intern.
5. No part of the project may look incomplete, copied, inconsistent or unprofessional during review. This includes the interface, folder structure, naming, written content, error messages, documentation and presentation.
6. Store API keys, passwords, database credentials and other secrets in environment variables. Never commit confidential values, .env files, private datasets or personal information to GitHub.
7. Include input validation, error handling, meaningful user feedback, loading states, empty states and recovery paths. The project must not fail silently or expose raw technical errors to users.



8. Follow a clean architecture and use reusable modules or components. Avoid large files, duplicated logic, unclear variable names and unnecessary dependencies.
9. Testing is mandatory. The intern must provide automated tests where practical and a manual test checklist for critical user flows or model scenarios.
10. The intern must be able to explain the complete project flow, code structure, technology choices, limitations, security considerations and future improvements during the final review.

4. Category-Specific Engineering Requirements

- Use a documented public, licensed or synthetic dataset. Never use private personal data without explicit authorization.
- Create a reproducible data split and prevent train-test leakage.
- Implement at least one simple baseline before using a more advanced model.
- Report metrics suitable for the task; accuracy alone is not sufficient for imbalanced classification.
- Include error analysis, model limitations and examples of failed predictions.
- Save model artifacts or provide reproducible training instructions.
- Use a lightweight interface or API so a reviewer can test the model without editing code.
- Do not make medical, legal, financial or hiring decisions. Present the output as decision support where relevant.

5. Weekly Submission Format

At the end of every week, the intern must submit the following evidence. Missing evidence means the weekly task is not considered complete even when the feature appears to work.

- GitHub repository link and the latest relevant commit or pull-request link.
- A concise weekly progress note stating completed work, pending work, blockers and decisions.
- Screenshots or a short screen recording that proves the implemented functionality.
- Updated README or technical notes whenever setup, architecture or project behavior changes.
- Testing evidence, including passed checks, known defects and the plan for fixing them.
- A clear list of next-week tasks mapped to the project plan.

6. Evaluation Framework

Area	Weight	What Will Be Reviewed	Minimum Standard
Engineering quality	25%	Code structure, readability, modularity, security, validation and reliability.	Professional
Functional completion	25%	Required features work correctly and satisfy the defined acceptance criteria.	Professional
Problem-solving depth	15%	The intern demonstrates reasoning, experimentation, debugging and justified technical decisions.	Professional



Area	Weight	What Will Be Reviewed	Minimum Standard
Professional presentation	15%	Interface quality, documentation, screenshots, demo quality and overall product polish.	Professional
Testing and evidence	10%	Automated tests, manual checks, metrics and defect handling are credible and reproducible.	Professional
GitHub discipline	10%	Repository naming, commits, branches, README, issue tracking and release quality.	Professional



Project 1: Devnexus DocuSense - Citation-Aware Document Question Answering Assistant

Project code	AI-01
Difficulty	Medium to challenging
Recommended duration	6 weeks
Primary domain	Natural Language Processing / RAG
Suggested repository	Devnexus-DocuSense
Recommended stack	Python, FastAPI or Streamlit, PyMuPDF, Sentence Transformers, FAISS or Chroma, optional LLM API

Project Brief

Build an intelligent assistant that accepts one or more PDF documents, indexes their content and answers user questions using only the uploaded material. Every answer must include a traceable reference to the relevant page or source section. The project should demonstrate document ingestion, semantic search, retrieval-augmented generation, confidence handling and a clean user experience.

Business Goal

Create a reliable research and knowledge-support tool that reduces the time required to locate information inside long documents while preventing unsupported or fabricated answers.

Required Final Product

- A working document-upload and question-answering application.
- Page-level or chunk-level citations displayed with every supported answer.
- A clear response when the requested information is not present in the documents.
- A searchable document index with delete or reset capability.
- A deployed demo and a documented local setup.

Functional Requirements

- Accept PDF documents and validate file type and file size.
- Extract text while preserving page numbers and document identity.
- Split content into meaningful overlapping chunks.
- Generate embeddings and store them in a vector index.
- Retrieve the most relevant chunks for each question.
- Generate a concise answer grounded only in retrieved context.
- Display source document name, page number and a short supporting excerpt.
- Maintain a simple session history without exposing private files.



Technical and Quality Requirements

- Compare at least two chunking or retrieval configurations and document the selected approach.
- Create a test set of at least 20 questions covering direct facts, multi-section questions and unanswerable questions.
- Measure retrieval precision or citation correctness using a documented manual or automated method.
- Use environment variables for all external API credentials.
- Log failures safely and prevent raw stack traces from appearing in the interface.
- Add unit tests for text extraction, chunk metadata and retrieval behavior.

Recommended Build Approach

1. Define supported document types, maximum file size, expected questions and privacy boundaries.
2. Build the ingestion pipeline first and verify extracted text and page metadata.
3. Implement chunking and semantic retrieval before adding response generation.
4. Add citation mapping so each answer can be traced to the retrieved source.
5. Create the interface, session controls and failure states.
6. Test with different document lengths, scanned-content limitations and unsupported questions.

Weekly Execution Plan

Week	Primary Goal	Required Tasks	Submission / Quality Gate
1	Research and architecture	• Write the problem statement, user stories and scope exclusions. • Select embedding model, vector store and interface framework. • Create repository, issue board, architecture diagram and sample-document test set.	• Approved plan • Repository initialized • 10 sample questions documented
2	Document ingestion	• Implement PDF validation and text extraction with page metadata. • Create configurable chunking with overlap. • Store extracted content in a structured format and add unit tests.	• Text verified on 5 PDFs • Chunk metadata correct • Tests passing
3	Semantic retrieval	• Generate embeddings and create the vector index. • Implement top-k retrieval and similarity thresholds. • Compare two retrieval settings and record findings.	• Relevant chunks returned • Comparison report • No duplicate indexing
4	Answer generation and citations	• Generate grounded answers from retrieved chunks. • Map answers to document and page references. • Implement an explicit no-answer response for insufficient evidence.	• Citations visible • Hallucination controls added • 20-question test executed



Week	Primary Goal	Required Tasks	Submission / Quality Gate
5	Product interface and quality	• Build upload, chat, source-view and reset flows. • Add loading, error, empty and unsupported-file states. • Improve response formatting and accessibility.	• Complete user flow • Responsive UI • No raw errors
6	Testing and release	• Complete evaluation, security review and performance checks. • Deploy the application and finalize documentation. • Record demo and prepare final report with limitations.	• Live deployment • README complete • Demo and report submitted

Final Deliverables

- Source code and complete GitHub history.
- Architecture diagram and documented data flow.
- Evaluation dataset and results.
- Deployed application link.
- README, technical report and 5-8 minute demo video.

Acceptance Criteria

- Answers are based only on indexed document content.
- At least 80% of the prepared factual questions return the correct supporting page.
- Unanswerable questions do not produce confident unsupported answers.
- The application handles invalid files and empty documents professionally.
- A reviewer can install and run the project using only the README.

Scope Boundaries

Do not build a general internet chatbot or a generic document management portal. OCR for scanned PDFs may be documented as an optional extension unless it is selected as a defined project feature.



Project 2: Devnexus TalentMatch - Resume Analyzer and Job Matching Engine

Project code	AI-02
Difficulty	Medium to challenging
Recommended duration	6 weeks
Primary domain	NLP / Semantic Matching
Suggested repository	Devnexus-TalentMatch
Recommended stack	Python, FastAPI or Streamlit, PyMuPDF, spaCy, Sentence Transformers, Scikit-learn, optional PostgreSQL

Project Brief

Build a professional tool that analyzes a candidate resume against a job description. It should extract relevant sections, identify technical and soft skills, calculate a transparent match score, highlight missing requirements and generate a structured improvement report. The tool must avoid using protected personal attributes and must explain how the score was produced.

Business Goal

Help candidates and recruiters understand job-resume alignment through transparent evidence rather than an unexplained single score.

Required Final Product

- Resume PDF upload and job-description input.
- Structured extraction of skills, education, experience and projects.
- Weighted match score with a visible scoring breakdown.
- Matched skills, missing skills and improvement recommendations.
- Downloadable professional analysis report.

Functional Requirements

- Validate and extract text from PDF resumes.
- Detect resume sections even when headings vary.
- Normalize skill names using a controlled skill dictionary.
- Use semantic similarity for experience and project relevance.
- Allow configurable weightings for skills, experience, education and keywords.
- Show evidence sentences that contributed to the score.
- Generate a report without exposing sensitive personal information.



Technical and Quality Requirements

- Use a curated test set of at least 15 resumes and 5 job descriptions, using public or synthetic data.
- Compare keyword-only matching with semantic matching.
- Document potential bias, false positives and scoring limitations.
- Exclude age, gender, photograph, nationality, religion and other protected attributes from scoring.
- Add tests for extraction, normalization and score calculations.
- Store uploaded files temporarily and delete them safely after processing.

Recommended Build Approach

1. Define a transparent scoring rubric before model implementation.
2. Build reliable document extraction and section parsing.
3. Create the skill normalization layer and keyword baseline.
4. Add semantic similarity and evidence extraction.
5. Design the score explanation and report output.
6. Evaluate consistency using multiple resumes for the same role.

Weekly Execution Plan

Week	Primary Goal	Required Tasks	Submission / Quality Gate
1	Requirements and scoring design	• Define target job types, supported resume format and ethical boundaries. • Create scoring rubric and synthetic/public evaluation set. • Initialize repository, data dictionary and architecture.	• Scoring rubric approved • Test data ready • Bias exclusions documented
2	Resume parsing	• Implement PDF extraction and section detection. • Extract skills, education, experience and projects. • Normalize common skill variants and test malformed files.	• 15 resumes parsed • Section accuracy reviewed • Validation implemented
3	Baseline matching	• Implement keyword and TF-IDF matching. • Create weighted score calculation. • Display matched and missing skills with evidence.	• Baseline score works • Breakdown visible • Unit tests added
4	Semantic matching	• Add sentence embeddings for experience and project relevance. • Compare semantic results with the baseline. • Calibrate thresholds and document trade-offs.	• Model comparison complete • Thresholds justified • Edge cases recorded
5	Interface and reporting	• Build upload, input, result and report-download flows. • Add privacy notice, loading	• Professional UI • PDF/HTML report generated • Private data not logged



Week	Primary Goal	Required Tasks	Submission / Quality Gate
		and error states. • Improve report wording and visual hierarchy.	
6	Evaluation and release	• Run final consistency and fairness checks. • Deploy the application and finalize README. • Prepare demo, evaluation summary and limitations.	• Live demo • Evaluation report • Final presentation ready

Final Deliverables

- Working matching application.
- Scoring rubric and model-comparison report.
- Public or synthetic evaluation dataset description.
- Downloadable analysis report.
- Repository, deployment, README and demo.

Acceptance Criteria

- The score breakdown is understandable and reproducible.
- Protected personal attributes are not used in the decision logic.
- The tool produces useful evidence for matched and missing requirements.
- The application processes typical one-to-three-page resumes without failure.
- All limitations and intended use are clearly disclosed.

Scope Boundaries

This project is a decision-support prototype, not an automated hiring decision system. It must not claim to determine candidate quality or replace human review.



Project 3: Devnexus MeetMind - Meeting Transcript Intelligence Tool

Project code	AI-03
Difficulty	Medium to challenging
Recommended duration	6 weeks
Primary domain	NLP / Information Extraction
Suggested repository	Devnexus-MeetMind
Recommended stack	Python, FastAPI or Streamlit, spaCy, Transformers, optional Whisper, SQLite or JSON storage

Project Brief

Develop a tool that converts a meeting transcript into a structured business summary. The output should separate the executive summary, decisions, action items, owners, due dates, risks and unresolved questions. A transcript-input version is mandatory; audio transcription may be added as an extension.

Business Goal

Reduce manual note-taking effort and produce consistent, reviewable meeting outcomes from unstructured conversations.

Required Final Product

- Transcript upload or paste input.
- Executive summary and topic-wise notes.
- Structured decisions and action items.
- Owner and deadline extraction when mentioned.
- Editable final output with export to PDF or Markdown.

Functional Requirements

- Clean speaker labels, timestamps and repeated filler content.
- Segment the transcript into coherent topics.
- Generate a concise summary with configurable length.
- Extract action statements, assigned owners and dates.
- Identify decisions, risks and open questions separately.
- Allow the user to edit extracted items before export.
- Preserve links to supporting transcript excerpts.

Technical and Quality Requirements

- Create a test set of at least 10 synthetic or public meeting transcripts.



- Evaluate action-item precision and missed-action rate.
- Use deterministic extraction rules as a baseline before advanced models.
- Prevent the model from inventing owners or deadlines not present in the transcript.
- Add tests for date normalization, speaker parsing and empty inputs.
- Document privacy controls and deletion behavior.

Recommended Build Approach

1. Define the exact output schema and examples.
2. Implement transcript cleaning and speaker-aware segmentation.
3. Create rule-based baseline extraction for actions and dates.
4. Add semantic summarization and classification.
5. Build evidence links and editable output.
6. Evaluate on different meeting styles and release the tool.

Weekly Execution Plan

Week	Primary Goal	Required Tasks	Submission / Quality Gate
1	Schema and dataset	• Define output JSON schema and final report structure. • Collect or create 10 safe transcripts with annotated actions and decisions. • Create repository and architecture plan.	• Dataset annotated • Schema approved • Privacy plan documented
2	Transcript preprocessing	• Parse speakers, timestamps and paragraph structure. • Normalize dates and remove obvious filler. • Add tests for malformed transcripts.	• Cleaning pipeline works • Speaker data preserved • Tests passing
3	Information extraction	• Implement action, owner, date and decision baseline rules. • Add topic segmentation and evidence spans. • Measure baseline extraction results.	• Baseline metrics reported • Evidence retained • False positives reviewed
4	Summarization intelligence	• Add semantic summary and topic labels. • Improve action and risk classification. • Implement safeguards against invented details.	• Summary quality reviewed • Unsupported fields blocked • Comparison documented
5	Editing and export experience	• Build transcript input and structured result interface. • Allow editing and removal of extracted items. • Export the final report to Markdown or PDF.	• End-to-end flow • Export works • Professional states and messages
6	Final evaluation	• Evaluate all 10 transcripts and document limitations.	• Evaluation complete • Deployment live



Week	Primary Goal	Required Tasks	Submission / Quality Gate
		• Deploy and complete security/privacy review. • Finalize README, report and demo.	• Final assets submitted

Final Deliverables

- Annotated transcript evaluation set.
- Structured extraction pipeline.
- Editable interface and export.
- Metrics and limitations report.
- Repository, deployment and demo.

Acceptance Criteria

- Action items include supporting transcript evidence.
- The system does not assign an owner or deadline unless supported by the transcript.
- Users can correct extracted content before export.
- The tool handles long transcripts without freezing or losing structure.
- The final report is suitable for professional internal use.

Scope Boundaries

Do not build a full meeting scheduling or employee-management system. Live conferencing and automatic recording are optional and should not replace the core transcript-intelligence requirements.



Project 4: Devnexus VisionInspect - Visual Defect Classification and Explainability

Project code	AI-04
Difficulty	Medium to challenging
Recommended duration	6 weeks
Primary domain	Computer Vision / Transfer Learning
Suggested repository	Devnexus-VisionInspect
Recommended stack	Python, PyTorch or TensorFlow, OpenCV, Albumentations, Streamlit or FastAPI, Grad-CAM optional

Project Brief

Create a computer-vision application that classifies product or surface images as normal or defective and, where supported by the dataset, identifies the defect category. The solution must include a documented dataset, transfer-learning experiments, model evaluation and an image-upload demo with confidence information.

Business Goal

Demonstrate an end-to-end visual inspection prototype that can support quality-control workflows and clearly communicate model confidence and limitations.

Required Final Product

- Image upload and prediction interface.
- Normal/defective classification with confidence.
- Defect category prediction where dataset labels permit.
- Evaluation dashboard with confusion matrix and class metrics.
- Visual explanation such as Grad-CAM or highlighted region as an advanced feature.

Functional Requirements

- Prepare a clean labeled dataset with reproducible train, validation and test splits.
- Apply appropriate resizing, normalization and augmentation.
- Train a transfer-learning baseline.
- Compare at least two architectures or training configurations.
- Display predicted class, confidence and model limitations.
- Reject unsupported file types and handle low-quality inputs.

Technical and Quality Requirements

- Prevent data leakage by splitting related images correctly.



- Report accuracy, precision, recall, F1-score and per-class confusion.
- Use early stopping, checkpointing and reproducible random seeds.
- Document class imbalance and mitigation.
- Provide a CPU-friendly inference path.
- Add tests for preprocessing and inference input validation.

Recommended Build Approach

1. Select a focused public dataset with manageable classes.
2. Audit labels, class balance and duplicate images.
3. Build preprocessing and baseline training pipeline.
4. Run controlled transfer-learning experiments.
5. Create the inference API or interface.
6. Evaluate failures and document production limitations.

Weekly Execution Plan

Week	Primary Goal	Required Tasks	Submission / Quality Gate
1	Dataset and experiment plan	• Select dataset and define defect classes. • Audit image counts, duplicates and label quality. • Create repository, experiment plan and data card.	• Dataset approved • Split strategy documented • Baseline target defined
2	Preprocessing pipeline	• Implement image loading, resizing and augmentation. • Create reproducible train, validation and test splits. • Visualize samples and add preprocessing tests.	• No data leakage • Sample audit complete • Pipeline reproducible
3	Baseline model	• Train first transfer-learning model. • Track loss and metrics. • Save best checkpoint and analyze confusion matrix.	• Baseline metrics recorded • Checkpoint saved • Major failure classes identified
4	Model improvement	• Train a second architecture or configuration. • Address imbalance and tune thresholds. • Add explainability visualization where feasible.	• Comparison complete • Best model selected • Improvements justified
5	Inference product	• Build image upload and prediction interface. • Add confidence display, image validation and helpful errors. • Optimize CPU inference and package model artifacts.	• End-to-end prediction • Latency measured • Invalid inputs handled
6	Release and reporting	• Run final test-set evaluation and error analysis. • Deploy the demo and	• Test metrics final • Deployment live • Model card complete



Week	Primary Goal	Required Tasks	Submission / Quality Gate
		complete README. • Prepare model card, report and video.	

Final Deliverables

- Clean training and inference code.
- Experiment logs and model comparison.
- Model checkpoint or reproducible download instructions.
- Evaluation report and model card.
- Deployed demo, README and video.

Acceptance Criteria

- Train, validation and test sets are reproducible and leakage-free.
- The selected model outperforms a documented baseline.
- Per-class performance and failure examples are reported.
- Inference works on a standard image without manual code changes.
- The project clearly states that the prototype requires domain validation before real industrial use.

Scope Boundaries

The project should focus on classification and optional visual explanation. Building a factory workflow, inventory system or production-control dashboard is outside scope.



Project 5: Devnexus ReviewGuard - Suspicious Review Detection with Explanations

Project code	AI-05
Difficulty	Medium
Recommended duration	6 weeks
Primary domain	NLP / Classification / Explainability
Suggested repository	Devnexus-ReviewGuard
Recommended stack	Python, Scikit-learn, spaCy or NLTK, SHAP or LIME, FastAPI or Streamlit

Project Brief

Build a model that analyzes written product or service reviews and estimates whether a review appears genuine or suspicious. The output should include the predicted label, probability, influential phrases and clear limitations. The project must emphasize evidence and model transparency rather than claiming certainty.

Business Goal

Provide a practical trust-and-safety prototype that helps reviewers inspect potentially manipulated content at scale.

Required Final Product

- Single-review and batch CSV analysis.
- Genuine/suspicious prediction with probability.
- Highlighted phrases or features influencing the result.
- Dataset and model comparison dashboard.
- Exportable batch results.

Functional Requirements

- Clean and normalize review text while retaining meaningful punctuation features.
- Create lexical, length, repetition and sentiment-related features.
- Train at least two classification models.
- Explain individual predictions using feature importance or local explanations.
- Support batch upload with row-level validation.
- Show uncertainty when confidence is near the decision threshold.

Technical and Quality Requirements

- Use a labeled public dataset and document licensing and limitations.
- Prevent duplicate reviews from leaking across splits.



- Report precision, recall, F1-score and class-specific errors.
- Calibrate or interpret probabilities carefully.
- Add tests for preprocessing, batch validation and model loading.
- Avoid presenting predictions as proof of fraud.

Recommended Build Approach

1. Audit the dataset and define what the labels actually represent.
2. Create a transparent feature-based baseline.
3. Train and compare classical text models.
4. Add explanation and uncertainty handling.
5. Build single and batch analysis flows.
6. Perform error analysis and publish limitations.

Weekly Execution Plan

Week	Primary Goal	Required Tasks	Submission / Quality Gate
1	Dataset audit	• Select dataset and document source, labels and constraints. • Remove duplicates and define split strategy. • Create repository and initial exploratory analysis.	• Data card ready • Leakage checks complete • Class balance documented
2	Feature pipeline	• Implement text cleaning and TF-IDF features. • Add selected behavioral text features. • Create preprocessing tests and baseline analysis.	• Feature pipeline reproducible • Tests passing • Feature rationale documented
3	Model comparison	• Train two or more classifiers. • Tune using validation data only. • Report precision, recall, F1 and confusion matrix.	• Comparison table complete • Best model selected • Overfitting checked
4	Explainability	• Add local explanation for individual predictions. • Highlight influential words or features. • Implement uncertain-result messaging.	• Explanation visible • No certainty claims • Threshold documented
5	Application	• Build single input and batch CSV workflows. • Add result filtering and export. • Implement professional errors and limits.	• Batch processing works • Export verified • UI polished
6	Final release	• Complete error analysis and model card. • Deploy and finalize documentation. • Record demo and prepare presentation.	• Deployment live • Model card complete • Final report submitted



Final Deliverables

- Dataset audit and data card.
- Training, evaluation and explainability code.
- Single and batch analysis application.
- Model card and error-analysis report.
- Repository, deployment and demo.

Acceptance Criteria

- Duplicate leakage checks are documented.
- Model performance is reported with class-specific metrics.
- Individual predictions include understandable evidence.
- Low-confidence predictions are clearly marked.
- The interface states that the result is a screening signal, not proof of misconduct.

Scope Boundaries

Do not create a review marketplace or product-management system. The core objective is detection, explanation and responsible evaluation.



Project 6: Devnexus RecoLab - Hybrid Recommendation Engine with Cold-Start Handling

Project code	AI-06
Difficulty	Medium to challenging
Recommended duration	6 weeks
Primary domain	Recommender Systems
Suggested repository	Devnexus-RecoLab
Recommended stack	Python, Pandas, Scikit-learn, Surprise or implicit, FastAPI or Streamlit, SQLite optional

Project Brief

Develop a hybrid recommendation engine that combines item-content similarity with user-interaction patterns. The system should recommend relevant items, explain why an item was suggested and provide a useful fallback for new users or new items with little interaction history.

Business Goal

Demonstrate practical personalization, offline evaluation and responsible handling of sparse data in a compact professional prototype.

Required Final Product

- Personalized top-N recommendations for existing users.
- Preference-based recommendations for new users.
- Content-similar alternatives for selected items.
- Simple explanation for each recommendation.
- Evaluation page comparing recommendation methods.

Functional Requirements

- Prepare user-item interactions and item metadata.
- Implement content-based recommendations.
- Implement collaborative filtering or implicit-feedback recommendations.
- Combine methods using a documented hybrid strategy.
- Handle new-user and new-item cold-start cases.
- Filter already-consumed items and remove duplicates.
- Display recommendation reasons and confidence-related information.

Technical and Quality Requirements

- Use train/test splitting suitable for recommendation data.



- Report Precision@K, Recall@K or NDCG@K.
- Compare at least three approaches: popularity baseline, content-based and collaborative/hybrid.
- Document sparsity, bias and popularity effects.
- Ensure reproducible results and saved model artifacts.
- Add tests for ranking, filtering and cold-start behavior.

Recommended Build Approach

1. Choose a focused public interaction dataset with item metadata.
2. Create a popularity baseline and evaluation protocol.
3. Build content and collaborative models independently.
4. Design and tune the hybrid strategy.
5. Add cold-start onboarding and explanations.
6. Build the demo and complete offline evaluation.

Weekly Execution Plan

Week	Primary Goal	Required Tasks	Submission / Quality Gate
1	Data and evaluation design	• Select dataset and define recommendation scenario. • Analyze sparsity, popularity and metadata quality. • Create repository, baseline plan and evaluation protocol.	• Dataset approved • Metrics selected • Popularity baseline defined
2	Content model	• Prepare item features and similarity model. • Implement item-to-item recommendations. • Create tests for duplicates and consumed-item filtering.	• Content recommendations working • Filtering correct • Examples reviewed
3	Collaborative model	• Build collaborative or implicit-feedback model. • Tune key parameters. • Evaluate against popularity baseline.	• Collaborative results measured • Baseline comparison • Artifacts saved
4	Hybrid and cold start	• Combine models using weighted or switching logic. • Create new-user preference onboarding. • Implement new-item and sparse-user fallback.	• Hybrid strategy documented • Cold start works • Metrics compared
5	Product experience	• Build user selection, preference input and recommendation views. • Add explanations and item-detail context. • Handle empty or insufficient-history cases.	• Professional flow • Reasons displayed • Edge cases covered
6	Final evaluation	• Run final ranking evaluation and bias analysis. • Deploy the demo and complete README.	• Evaluation final • Deployment live • Report submitted



Week	Primary Goal	Required Tasks	Submission / Quality Gate
		• Prepare technical report and presentation.	

Final Deliverables

- Prepared data pipeline and reproducible evaluation.
- Popularity, content, collaborative and hybrid models.
- Interactive recommendation demo.
- Metrics, bias and cold-start report.
- Repository, deployment, README and video.

Acceptance Criteria

- Hybrid recommendations outperform the popularity baseline on at least one selected ranking metric.
- Cold-start flows return sensible recommendations without fake user history.
- Already-consumed items are excluded where appropriate.
- Recommendation explanations are meaningful and not misleading.
- Evaluation can be reproduced from the repository.

Scope Boundaries

Do not build a complete e-commerce or content-management platform. Use a compact demo catalog and focus on recommendation quality, evaluation and explanation.



Final Submission Checklist

1. The repository name follows the Devnexus naming requirement and contains no confidential data.
2. The default branch contains a stable, tested and deployable version of the project.
3. The README is complete and allows a reviewer to set up the project without private assistance.
4. The project includes clear screenshots, a short architecture diagram and a feature demonstration.
5. All required weekly tasks and acceptance criteria have been completed or transparently documented as limitations.
6. The live deployment works on a fresh browser or environment and does not depend on local-only files.
7. Error messages, empty states, loading states and invalid-input scenarios have been tested.
8. The intern has prepared a 5-8 minute final demo and can answer technical questions about every major module.
9. The final report includes objectives, implementation, testing, results, challenges, limitations and future improvements.
10. The project is visually and technically professional enough to be included in an internship portfolio.

Final review principle: The Devnexus AI / ML Internship evaluates ownership, engineering judgment and professional execution. A smaller project completed with strong quality is preferred over a larger project that is copied, unstable or poorly documented.

Official Contact

Company	Devnexus Digital Solutions
Email	devnexus.support@gmail.com
Phone	+92 303 0111550
Internship portal	careers-devnexus.site
Work mode	Remote (Only)